

Discrete Structures

Spring 2026 – Week 4

Algorithms

Problems and Algorithms

- In many **domains** there are common problems where we are given some **input** and we must produce an **output** with certain **properties**.
- First, we clearly define the problem by describing what the **input** is and what **output** we want.
- Then, we **solve** the **problem** by writing clear, **step-by-step** instructions that take the **input** and give the correct **output**.
- These **step-by-step** instructions are called an **algorithm**.

Algorithms



Abu Ja'far Mohammed Ibin Musa Al-Khowarizmi
(780-850)

- ***Definition:*** An **algorithm** is a **finite** set of **precise** instructions for performing a **computation** or for solving a **problem**.
- ***Example:*** Describe an algorithm for finding the **maximum value** in a finite sequence of **integers**.

Algorithms

- **Solution: Perform the following steps:**

1. Set the first number as the temporary maximum.
2. Compare each next number to the temporary maximum.
3. If a number is larger, make it the new temporary maximum.
4. Repeat until all numbers are checked.
5. The temporary maximum is the largest number.

Specifying Algorithms

- **Algorithms** can be specified in different ways. Their steps can be described in **English** or in **pseudocode**.
- **Pseudocode** is an intermediate step between an English language description of the steps and a coding of these steps using a **programming language**.
- **Programmers** use **pseudocode** to later write the same steps in any programming language, like **C**, **Java**, or **Python**.
- **Pseudocode** also helps us understand **how much time an algorithm takes** to solve a problem, without worrying about which **programming language** is used.

Properties of Algorithms

- **Input:** An algorithm takes some **input values** to work on.
- **Output:** After processing the **input**, the algorithm gives an **output**, which is the answer to the **problem**.
- **Correctness:** An algorithm should produce the **correct output values** for each set of input values.
- **Finiteness:** An algorithm should produce the **output after a finite number of steps** for any input.
- **Effectiveness:** Each step of the **algorithm** should be **simple and doable**, and take a **reasonable** amount of time.
- **Generality:** The algorithm should work for **all similar problems**, not just one specific case.

Finding the Maximum Element in a Finite Sequence

- The algorithm in pseudocode:
- Start with the first number as the maximum.
- Look at each remaining number:
- If the number is bigger than the current maximum, make it the new maximum.
- After checking all numbers, the maximum is the largest number.

```
procedure max( $a_1, a_2, \dots, a_n$  : integers)  
   $max := a_1$   
  for  $i := 2$  to  $n$   
    if  $max < a_i$  then  $max := a_i$   
  return  $max$  { $max$  is the largest element}
```

- Does this algorithm have all the properties listed on the previous slide?

Some Example Algorithm Problems

- Classes of problems to be studied in this section.
 1. *Searching Problems:* finding the position of a particular element in a list.
 2. *Sorting problems:* putting the elements of a list into increasing order (smallest to largest/Ascending).

Searching Problems

You are given:

A list of different elements: $a_1, a_2, \dots, a_n$

A value x that you want to find

The goal is:

- To check **whether x is in the list or not**.
- If **x is found, the answer is the position (index)** where x is located in the list.
- If **x is not found, the answer is 0**, which means the value is not in the list.

Linear Search Algorithm

- The **linear search** algorithm- Finds the position of a number in a list by checking **each element one by one**, starting at the **beginning**.
- **Steps:**
 1. Start with the first element.
 2. Compare it to the number you are looking for.
 3. If it matches, return its position.
 4. If not, move to the next element.
 5. Repeat until you find the number or reach the end of the list.
 6. If the number is not found, return 0.

Linear Search Algorithm

List: [4, 7, 2, 9, 5]

Search for 9 → check 4, 7, 2, then 9 → found at position 4

Search for 6 → check all elements → not found → return 0

```
i := 1
while i ≤ n and a[i] ≠ x
    i := i + 1
if i ≤ n then
    return i // found
else
    return 0 // not found
```

Binary Search Algorithm

- Find the position of a number in a **sorted list** by repeatedly **cutting the list in half**.

Steps:

1. Check the **middle element** of the list.
2. If it equals the number, **return its position**.
3. If it is smaller than the number, search the **left half**.
4. If it is larger, search the **right half**.
5. Repeat steps 1-4 until the number is **found or the list is empty**.
6. If the number is **not found**, **return 0**.

Binary Search Algorithm

```
procedure binary search( $x$ : integer,  $a_1, a_2, \dots, a_n$ : increasing integers)
   $i := 1$  { $i$  is the left endpoint of interval}
   $j := n$  { $j$  is right endpoint of interval}
  while  $i < j$ 
     $m := \lfloor (i + j) / 2 \rfloor$ 
    if  $x > a_m$  then  $i := m + 1$ 
    else  $j := m$ 
  if  $x = a_i$  then  $location := i$ 
  else  $location := 0$ 
  return  $location$  {location is the subscript  $i$  of the term  $a_i$  equal to  $x$ ,
    or 0 if  $x$  is not found}
```

Binary Search Algorithm

Example: The steps taken by a binary search for 19 in the list:

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

1. The list has **16** elements, so the **midpoint** is **8**. The value in the **8th** position is **10**. Since **19 > 10**, further search is restricted to positions **9** through **16**.

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

2. The midpoint of the list (positions **9** through **16**) is now the **12th** position with a value of **16**. Since **19 > 16**, further search is restricted to the **13th** position and above.

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

Binary Search Algorithm

3. The **midpoint** of the current list is now the **14th** position with a value of **19**. Since **19 > 19**, further search is restricted to the left half from the **13th -14th** positions .

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

4. The **midpoint** of the current list is now the **13th** position with a value of **18**. Since **19 > 18**, search is restricted to the portion from the **14th** position through the **14th**.

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

5. Now the list has a single element and the loop ends. Since **19=19**, the location **14** is returned.

Sorting Algorithms

Sorting

- **Sorting** means arranging the items in a list in a **proper order**, such as:
 1. Numbers from **small to large**
 2. Words in **alphabetical order**
- **Sorting is an important problem because:**
- Computers spend a **lot of time and power** sorting data.
- Many real-life applications need data in a **specific order**, for example:
 1. Customer names in **alphabetical order**
 2. Product or part numbers in **numeric order**
 3. Records in **large databases** arranged neatly

Bubble Sort

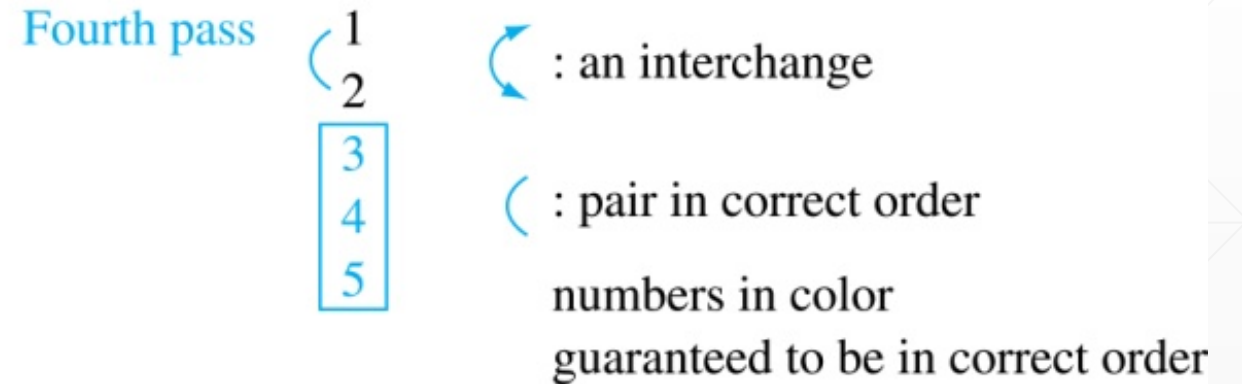
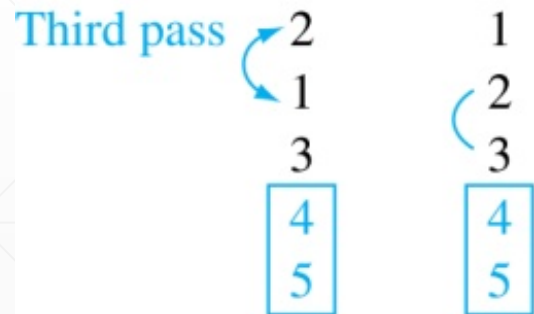
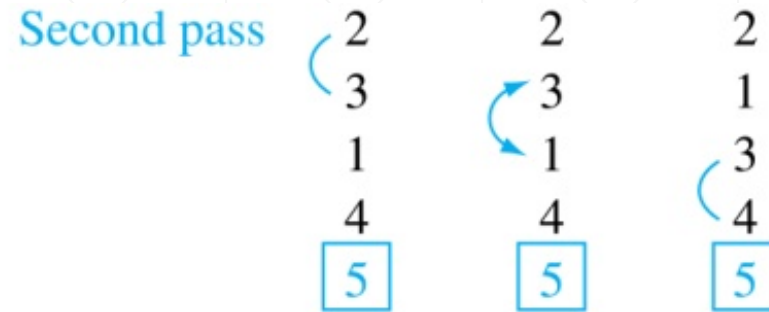
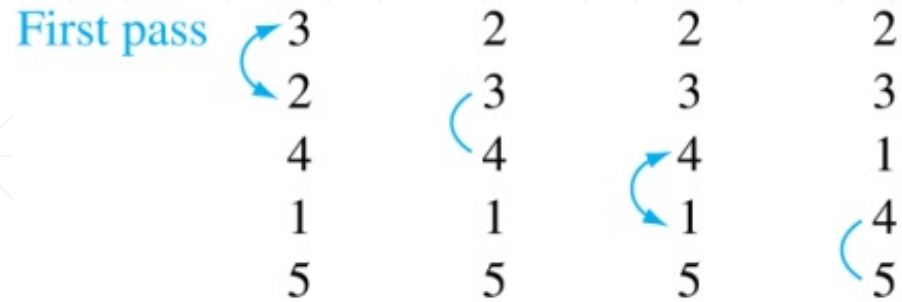
- *Bubble sort* makes multiple passes through a list. Every pair of elements that are found to be out of order are interchanged.

```
procedure bubblesort( $a_1, \dots, a_n$  : real numbers  
                    with  $n \geq 2$ )  
  for  $i := 1$  to  $n - 1$   
    for  $j := 1$  to  $n - i$   
      if  $a_j > a_{j+1}$  then interchange  $a_j$  and  $a_{j+1}$   
  { $a_1, \dots, a_n$  is now in increasing order}
```

Bubble Sort

- **Example:** Show the steps of bubble sort with 3 2 4 1 5
- In the first pass, the **largest number** moves to its correct place.
- In the second pass, the **second largest number** moves to its correct place.
- Each new pass puts **one more number** in the correct position.

Bubble Sort



Bubble Sort – Illustration

6 5 3 1 8 7 2 4

https://en.wikipedia.org/wiki/Bubble_sort#/media/File:Bubble-sort-example-300px.gif

Insertion Sort

- **Insertion sort** begins with the **2nd element**. It compares the **2nd element** with the **1st** and puts it before the **first** if it is not larger.
- Next the **3rd** element is put into the correct position among the first **3** elements.
- In each subsequent pass, the **$n+1^{\text{st}}$** element is put into its correct position among the first **$n+1$** elements.
- **Linear search** is used to find the correct position.

Insertion Sort

- **Example:** Show all the steps of insertion sort with the input:

- 3 2 4 1 5

- i. 2 3 4 1 5 *(first two positions are interchanged)*

- ii. 2 3 4 1 5 *(third element remains in its position)*

- iii. 1 2 3 4 5 *(fourth is placed at beginning)*

- iv. 1 2 3 4 5 *(fifth element remains in its position)*

Continue on Next Slide

Insertion Sort (Further Explanation)

Step 1: Start with the second element (2)

- Compare 2 with the first element 3.

Since $2 < 3$, swap them.

List now: 2 3 4 1 5

Step 2: Take the third element (4)

- Compare 4 with elements before it: 3 and 2.
- $4 > 3$ → it's already in the right position.

List now: 2 3 4 1 5

Step 3: Take the fourth element (1)

- Compare 1 with 4 → $1 < 4$, move 4 right
- Compare 1 with 3 → $1 < 3$, move 3 right
- Compare 1 with 2 → $1 < 2$, move 2 right
- Place 1 at the beginning

List now: 1 2 3 4 5

Step 4: Take the fifth element (5)

- Compare 5 with 4 → $5 > 4$, so it stays in place

Final sorted list: 1 2 3 4 5

Insertion Sort – Illustration

6 5 3 1 8 7 2 4

https://en.wikipedia.org/wiki/Insertion_sort#/media/File:Insertion-sort-example-300px.gif

Insertion Sort

```
procedure insertion sort
  ( $a_1, \dots, a_n$ :
    real numbers with  $n \geq 2$ )
  for  $j := 2$  to  $n$ 
     $i := 1$ 
    while  $a_j > a_i$ 
       $i := i + 1$ 
     $m := a_j$ 
    for  $k := 0$  to  $j - i - 1$ 
       $a_{j-k} := a_{j-k-1}$ 
     $a_i := m$ 
  {Now  $a_1, \dots, a_n$  is in increasing order}
```

Complexity of Algorithms

- **Why do we even need “complexity”?**
- Let's say you have **two algorithms** that solve the same problem.

- **Example:**

Finding a number in a list of 1,000,000 numbers.

- Algorithm A checks **every number one by one**
- Algorithm B keeps **halving the list**
- Both work.

But one is **much faster**.

- **Algorithm Complexity tells us which algorithm is better as input size grows**

Complexity of Algorithms

- ▪ **How can the efficiency of an algorithm be analyzed?**
 - By Measuring algorithm efficiency as **input size grows**
- **Algorithm Complexity** = How much **time** or **memory** an algorithm needs when input size increases.
 - **Two main types:**
 - **Time Complexity** [Number of operations required/ How long an algorithm takes to run]
 - **Space Complexity** [How much extra memory it uses]

Complexity of Algorithms

- We use **n** to represent input size.
- **Examples:**
 - Searching in a list of **n** elements
 - Sorting **n** numbers

Big-O is a way to **describe growth**.

“How fast does work increase when **n** increases?”

Big-O	Meaning
$O(1)$	Constant time
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n^2)$	Quadratic

- **$O(1)$ – Constant Time**

- Time does not depend on n .

- **Example:**

- Checking if a number is even

- Even if:

- $n = 10$
 - $n = 1,000,000$
 - It will work on $n \% 2 = 0$

- Means the no. of operation doesnot change with input size n

- **$O(n)$ – Linear Time**

- Time increases directly with n .

- **Example: Linear Search**

- **Best case:** element found at first position
 - **Worst case:** element found at last position or not found

- Worst case → check all n elements.

- Time Complexity = $O(n)$**

- **$O(\log n)$ – Logarithmic Time**

- This is very fast.

- Each step cuts the problem in half.

- **Example: Binary Search**

- **Steps:** {But array must be sorted.}

1. Check middle element
2. If not found, search half of the array
3. Repeat

- For 16 elements:

- Step 1 $\rightarrow$ 8 left
- Step 2 $\rightarrow$ 4
- Step 3 $\rightarrow$ 2
- Step 4 $\rightarrow$ 1

- **Only 4 steps!**

- **$O(n^2)$ – Quadratic Time**

- Used when there are nested loops.

- **Example: Bubble Sort & Insertion Sort**

- Outer loop runs n times
- Inner loop runs n times

- **Total operations $\approx n \times n = n^2$**

- How time grows?

For $n=10$, operations=100

For $n=100$, operations=10,000

- Why this happens?

For Each element, you again loop for each element.